



CS318

Computer Organization

EELU | Fall 2026-2027



Lecture 1

CS318 Computer Organization

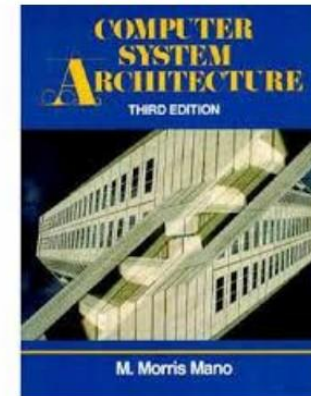


Course Administration

Lectures: Sat & Tues 11:00 – 12:00 pm
Sections : From 0-:00 to 0-:00 am

- **Exams:** 2 Quizzes (20%), Midterm (20%), Activities (10) and final (50%).
- Activities [**Homework** (...%). **Attendance** and **Participation** (...%)].
- **Slides:** should be available on: www.eelu.edu.eg
- **Office hours:** Monday 00.00-00.00 pm, or by appointment at **EELU- Ain Shams Centre**.
- **TA:** WBA.

Textbook:



Computer System
Architecture (3rd Edition)
by M. Morris Mano
Published by Pearson
Prentice Hall, 1993.

Acknowledgement

- This lecture note is modified from Engin112: Digital Design by Prof. Maciej Ciesielski, Prof. Tilman Wolf, University of Massachusetts Amherst and original slide from publisher

Computer Architecture & Computer Organization

Computer architecture is the programmer's view, encompassing the logical, Instruction Set Architecture (ISA) such as instruction sets, registers, data types, and addressing modes.

In contrast,

computer organization is the implementation view, focusing on the physical, low-level details like circuit design, control signals, clock frequency, and the interconnection of components that realize the architecture.

The analogy is that **architecture** is the abstract design of a building, while **organization** is the physical assembly of its parts.



Architecture vs Engineering

Computer Architecture & Computer Organization

■ Computer organization

- Encompasses all physical aspects of computer systems.
- E.g., circuit (Logic circuit) design, control signals, memory types.
- How does a computer work?

■ Computer architecture

- Logical aspects of system implementation as seen by the programmer.
- E.g., instruction sets, instruction formats, data types, addressing modes.
- How do I design a computer?

Computer Architecture & Computer Organization & Computer Design

Architecture, organization, and design

- These three confusing computer terms are explained here:

Term	Computer architecture	Computer organization	Computer design
✓ Definition	The structure and behavior of the functional modules	The connection of the hardware components	The development of the hardware
✓ Examples	Addressing technique, instruction set	Memory technology, control signals	Processor and memory hardware

So, having a multiply instruction in a computer is an architecture issue but how to implement the instruction is an organization issue and how fast to process the multiplication is a design issue

Architecture (the "What"): Determines the instruction set and overall functional characteristics.

Organization (the "How"): Details the hardware implementation and structural connections to achieve the architecture.

Design (the "Realization"): The overall process of bringing a specific computer system into existence by making choices in both architecture and organization
(*Designing a specific CPU model based on the x86 architecture and its specific organizational implementation.*)

Class Overview

- All 3 subjects associated with computer hardware in this book
 - ◆ Computer Organization(Chap 1, 2, 3, 4)
 - H/W components operation/connection.
 - Various digital components used in the organization and design of digital computer.
 - ◆ Computer Design(Chap 5, 6, 7)
 - H/W Design/Implementation.
 - The steps that a designer must go through to design and program an elementary digital computer(Chap. 6 : program = ISA)
 - ◆ Computer Architecture(Chap 6, 8, 9, 11, 12)
 - Structure and behavior of the computer as seen by the user
 - » Information format, Instruction set, memory addressing : S/W = ISA
 - » CPU, I/O, Memory : H/W
 - Chapter in detail
 - » Chap. 6 : ISA
 - » Chap. 8 and 9 : CPU
 - » Chap. 11 : I/O
 - » Chap. 12 : Memory

Computer Organization & Computer Architecture

1-1 Digital Computers

■ Digital Computer = H/W + S/W

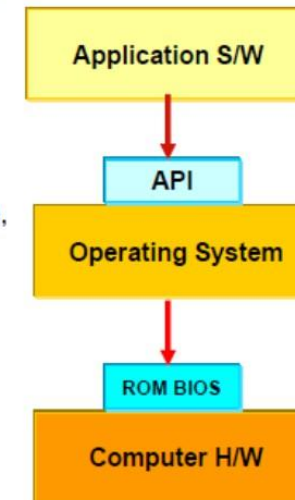
◆ Digital

- implies that the information in the computer is represented by variables that take a limited number of **discrete values**.
- the decimal digits 0, 1, 2, ..., 9, provide 10 discrete values, but digital computers function **more reliably** if only **two states** are used.
- because of the physical restriction of components, and because human logic tends to be **binary (true/false, yes/no)**, digital component are further constrained to take **only two values** and are said to be **binary**.

◆ Bit = binary digit : 0/1

◆ Program(S/W)

- A sequence of instruction
- S/W = Program + Data
 - » The data that are manipulated by the program constitute the data base
- Application S/W = DB, word processor, Spread Sheet
- System S/W = OS, Firmware, Compiler, Device Driver



1-1 Digital Computers

1-10

continued

■ Computer Hardware(H/W)

◆ CPU

◆ Memory

- Program Memory(ROM)
- Data Memory(RAM)

◆ I/O Device

- Interface: 8251 SIO, 8255 PIO, 6845 CRTC, 8272 FDC, 8237 DMAC, 8279 KDI
- Input Device: Keyboard, Mouse, Scanner
- Output Device: Printer, Plotter, Display
- Storage Device(I/O): FDD, HDD, MOD

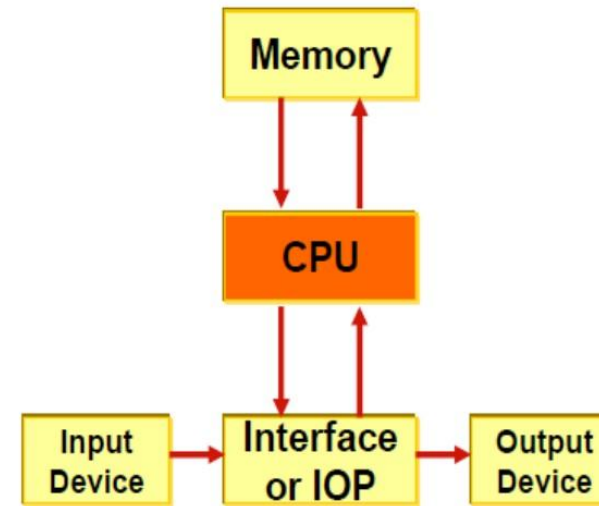


Figure 1-1 Block Diagram of a digital Computer

1-2 Logic Gates

1-11

■ ADC(Analog to Digital Conversion)

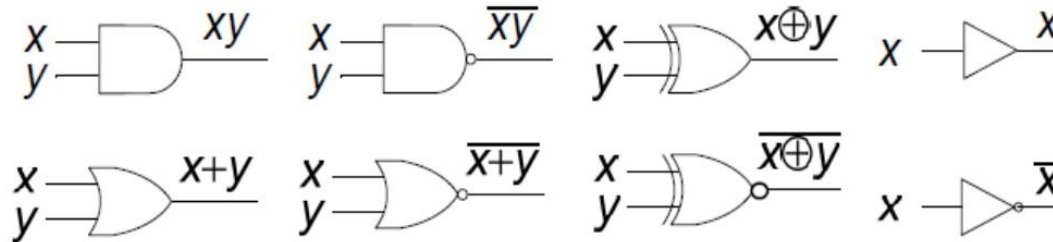
◆ Signal $\longrightarrow$ Physical Quantity $\longrightarrow$ Binary Information

$V, A, F, \text{거리}$ Discrete Value

$\left\{ \begin{array}{l} 0 : 0.5v \\ 1 : 3v \end{array} \right.$

■ Gate

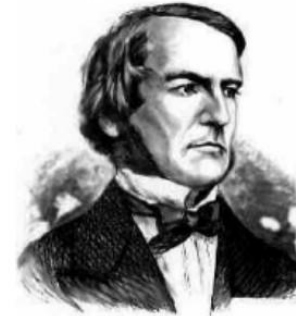
- ◆ The manipulation of binary information is done by logic circuit called "gate".
- ◆ Blocks of H/W that produce signals of binary 1 or 0 when input logic requirements are satisfied.
- ◆ Digital Logic Gates : Fig. 1-2
 - AND, OR, INVERTER, BUFFER, NAND, NOR, XOR, XNOR



1-3 Boolean Algebra

1-12

- Boolean Algebra
 - ◆ Deals with binary variable (A, B, x, y: T/F or 1/0) + logic operation (AND, OR, NOT...)
- Boolean Function: *variable + operation*
 - ◆ $F(x, y, z) = x + y'z$
- George Boole
 - ◆ Born: 2 Nov 1815 in Lincoln, Lincolnshire, England
 - ◆ Died: 8 Dec 1864 in Ballintemple, County Cork, Ireland



1-3 Boolean Algebra

1-13

- Boolean Function: variable + operation

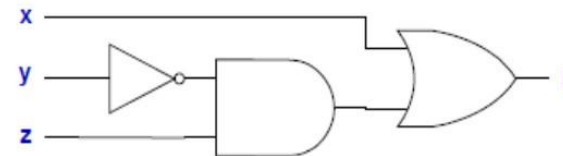
◆ $F(x, y, z) = x + y'z$

- Truth Table: Fig. 1-3(a)
Relationship between a function and variable

2nd Combination
Variable n = 3

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

- Logic Diagram: Fig. 1-3(b)
Algebraic Expression $\rightarrow$
Logic Diagram(gates로 표현)



- Purpose of Boolean Algebra
 - ◆ To facilitate the analysis and design of digital circuit
- Boolean function = **Algebraic form** = convenient tool
 - ◆ Truth table (relationship between binary variables : Fig 1-3a) → Algebraic form
 - ◆ Logic diagram (input-output relationship : Fig. 1-3b) → Algebraic form
 - ◆ Find simpler circuits for the same function : *by using Boolean algebra rules*
- Boolean Algebra Rule : Tab. 1-1
 - **Operation with 0 and 1:** $x + 0 = x$, $x + 1 = 1$, $x \cdot 1 = x$, $x \cdot 0 = 0$
 - **Idempotent Law:** $x + x = x$, $x \cdot x = x$
 - **Complementary Law:** $x + x' = 1$, $x \cdot x' = 0$
 - **Commutative Law:** $x + y = y + x$, $x \cdot y = y \cdot x$
 - **Associative Law:** $x + (y + z) = (x + y) + z$, $x \cdot (y \cdot z) = (x \cdot y) \cdot z$
 - **Distributive Law:** $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$, $x + (y \cdot z) = (x + y) \cdot (x + z)$
 - **DeMorgan's Law:** $(x + y)' = x' \cdot y'$, $(x \cdot y)' = x' + y'$
 - General Form: $(x_1 + x_2 + x_3 + \dots x_n)' = x_1' \cdot x_2' \cdot x_3' \cdot \dots x_n'$
 $(x_1 \cdot x_2 \cdot x_3 \cdot \dots x_n)' = x_1' + x_2' + x_3' + \dots x_n'$

$$(A \cup B)^c = A^c \cap B^c$$

1-4 Map Simplification

1-10

- Karnaugh Map(K-Map)
 - ◆ Map method for simplifying Boolean expressions
- Minterm / Maxterm
 - ◆ Minterm : n variables **product** ($x=1, x'=0$)
 - ◆ Maxterm : n variables **sum** ($x=0, x'=1$)
- 2 variables example

x	y	Minterm	Maxterm
0	0	$x'y'$ m_0	$x + y$ M_0
0	1	$x'y$ m_1	$x + y'$ M_1
1	0	$x y'$ m_2	$x' + y$ M_2
1	1	$x y$ m_3	$x' + y'$ M_3

$$\begin{aligned}
 \text{■ } F &= \underbrace{x'y}_{m_1} + \underbrace{xy}_{m_3} \\
 &= \Sigma(1,3) \quad (m_1 + m_3) \\
 &= \Pi(0,2) \quad (\text{Complement} = M_0 \cdot M_2)
 \end{aligned}$$

$$\underbrace{m_0 + m_1 + m_2 + m_3} \quad \underbrace{M_0 \cdot M_1 \cdot M_2 \cdot M_3}$$

■ Map

◆ 2 variables

	B	
	0	1
A {	2	3

◆ 3 variables

		B		
	0	1	3	2
A {	4	5	7	6
	C			

◆ 4 variables

		C		
	0	1	3	2
	4	5	7	6
A {	12	13	15	14
	8	9	11	10
	D			
	B			

◆ 5 variables

				C				
0	1	3	2	6	7	5	4	
8	9	11	10	14	15	13	12	
A {	24	25	27	26	30	31	29	28
	16	17	19	18	22	23	21	20
		E		D		F		

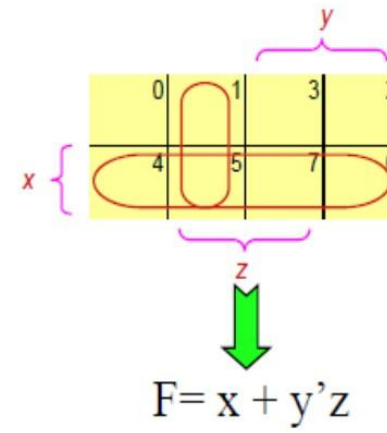
■ [예제] $F = x + y'z$

(1) Truth Table

x	y	z	F	Minterm
0	0	0	0	m_0
0	0	1	1	m_1
0	1	0	0	m_2
0	1	1	0	m_3
1	0	0	1	m_4
1	0	1	1	m_5
1	1	0	1	m_6
1	1	1	1	m_7

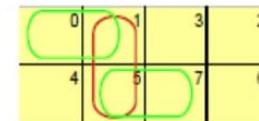
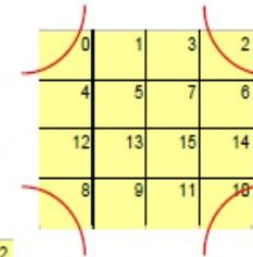
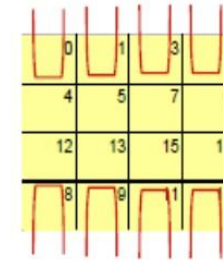
(2) $F(x, y, z) = \Sigma(1, 4, 5, 6, 7)$

(3)



■ Adjacent Square

- ◆ Number of square = 2^n (2, 4, 8,)
- ◆ The squares at the extreme ends of the same horizontal row are to be considered adjacent
- ◆ The same applies to the top and bottom squares of a column
- ◆ The four corner squares of a map must be considered to be adjacent
- ◆ Groups of combined adjacent squares may share one or more squares with one or more group



■ [예제] $F(A, B, C) = \Sigma(3, 4, 6, 7)$

◆ $F = AC' + BC$

	0	1	3	2
A {	4	5	7	6

■ [예제] $F(A, B, C) = \Sigma(0, 2, 4, 5, 6)$

◆ $F = C' + AB'$

	0	1	3	2
A {	4	5	7	6

■ [예제] $F(A, B, C, D) = \Sigma(0, 1, 2, 6, 8, 9, 10)$

◆ $F = B'D' + B'C' + A'CD'$

	0	1	3	2
A {	4	5	7	6
	12	13	15	14
	8	9	11	10

■ Product-of-Sums Simplification

$F(A, B, C, D) = \Sigma(0, 1, 2, 5, 8, 9, 10)$

$F = B'D' + B'C' + A'CD'$ Sum of product

$F' = AB + CD + BD'$ (square marked 0's)

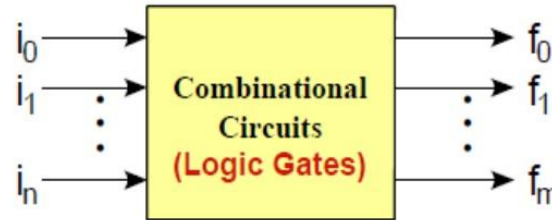
$F''(F) = (A' + B')(C' + D')(B' + D)$ Product of Sum

	0	1	3	2
A {	4	5	7	6
	12	13	15	14
	8	9	11	10

1-5 Combinational Circuits

■ Combinational Circuits

- ◆ A connected arrangement of *logic gates* with a set of inputs and outputs
- ◆ *Fig. 1-15 Block diagram of a combinational circuit*



■ Analysis

- ◆ Logic circuits diagram  Boolean function or Truth table

■ Design (Analysis의 반대)

Experience

- ◆ 1. The Problem is stated
- ◆ 2. I/O variables are assigned
- ◆ 3. Truth table(I/O relation)
- ◆ 4. Simplified Boolean Function(Map 과 Boolean 대수 이용)
- ◆ 5. Logic circuit diagram

■ Design Example : Full Adder

- ◆ 1. Full adder is a combinational circuits that forms the arithmetic sum of three input bit (*Carry considered*)
- ◆ 2. 3 Input(x, y, z), 2 Output(S: sum, C: carry)
- ◆ 3. Truth Table

Input			Output	
x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

◆ 4. Simplification

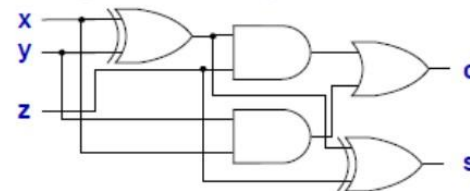
		y	
	0	1	3
x {	4	5	7
		z	

$$\begin{aligned}
 C &= xy'z + x'y'z + xy \\
 &= z(xy' + x'y) + xy \\
 &= z(x \oplus y) + xy
 \end{aligned}$$

		y	
	0	1	3
x {	4	5	7
		z	

$$\begin{aligned}
 S &= xy'z' + x'y'z + xy'z + x'yz' \\
 &= z'(xy' + x'y) + z(xy' + xy) \\
 &= z'(x \oplus y) + z(x \oplus y)' \\
 &= a'b + ab' \text{ (let } a=z, b=x \oplus y) \\
 &= x \oplus y \oplus z
 \end{aligned}$$

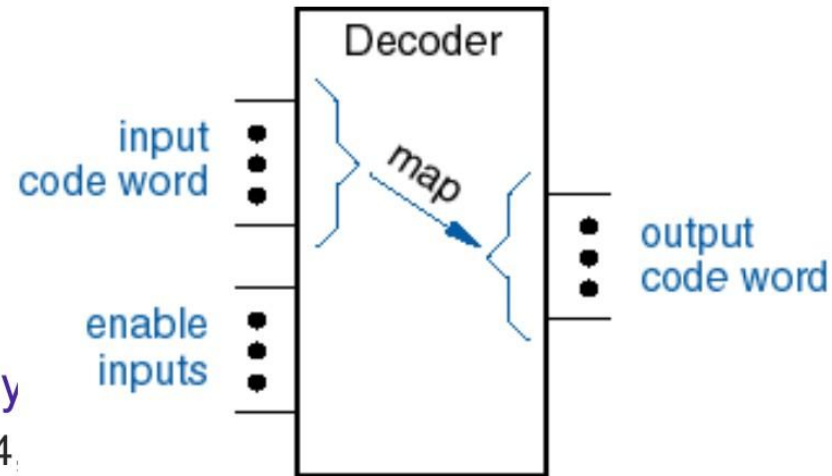
◆ 5. Logic circuit diagram



$$\begin{aligned}
 (x \oplus y)' &= (xy' + x'y)' \\
 &= (x' + y)(x + y') \\
 &= x'y' + xy \\
 &= x'y' + xy
 \end{aligned}$$

Decoders

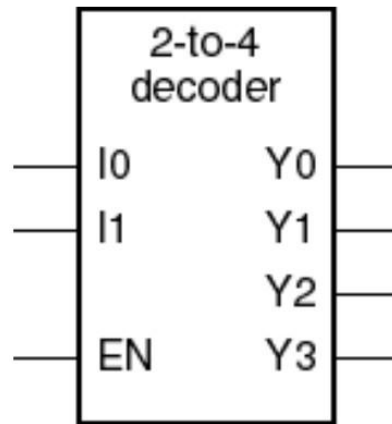
■ General decoder structure



■ Typically

- 2-to-4

Binary 2-to-4 decoder

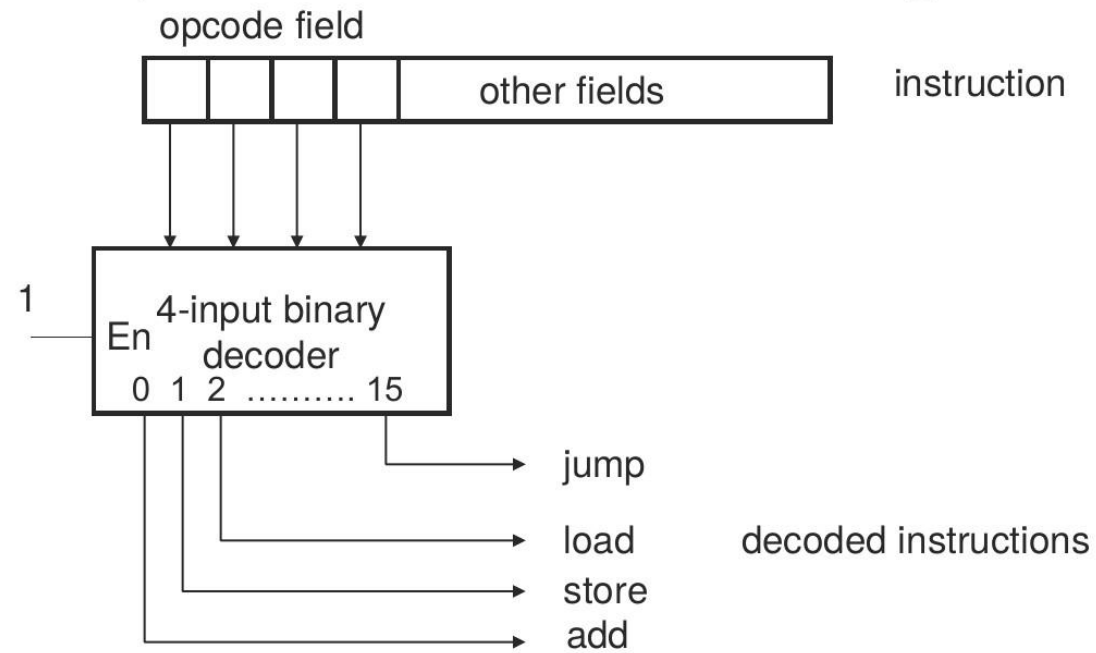


Inputs			Outputs			
EN	I1	I0	Y3	Y2	Y1	Y0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

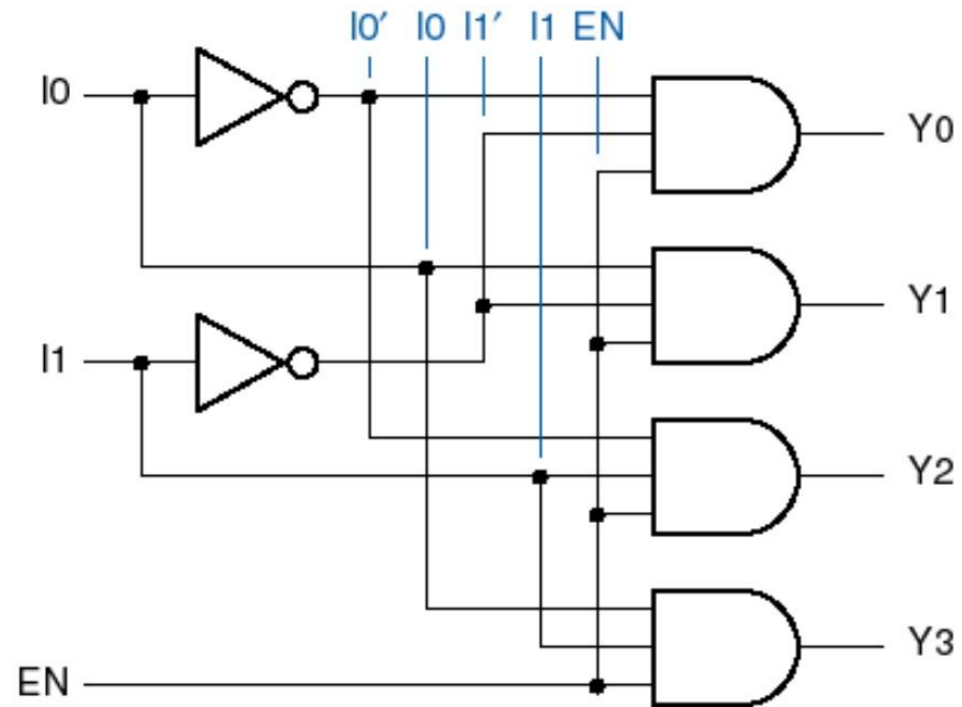
Note: “x” = (don’t care) cases.

Decoder Use – Operation Decoding

•Microprocessor instruction decoding



2-to-4-decoder logic diagram



3-input Binary Decoder

E	x_2	x_1	x_0	x	y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0
1	0	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	1	0	0	0	0	0	0	1	0
1	0	1	0	2	0	0	0	0	0	1	0	0
1	0	1	1	3	0	0	0	0	1	0	0	0
1	1	0	0	4	0	0	0	1	0	0	0	0
1	1	0	1	5	0	0	1	0	0	0	0	0
1	1	1	0	6	0	1	0	0	0	0	0	0
1	1	1	1	7	1	0	0	0	0	0	0	0
0	-	-	-	-	0	0	0	0	0	0	0	0

Inputs: $\underline{x} = (x_2, x_1, x_0)$, with x_i in $\{0,1\}$ and E in $\{0,1\}$

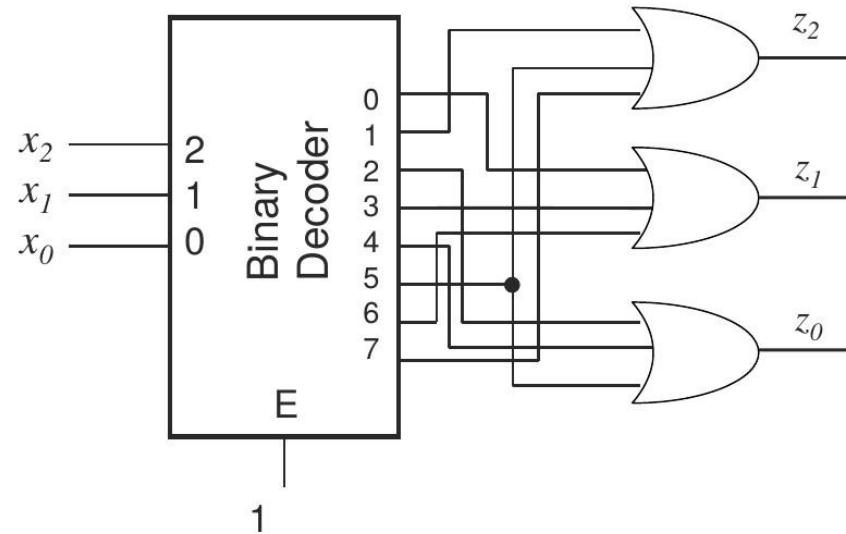
Outputs: $\underline{y} = (y_7, y_6, y_5, \dots, y_1, y_0)$ with y_i in $\{0,1\}$

Function: $y_i = E \cdot m_i(\underline{x})$, $i = 0, 1, \dots, 7$

Implementing functions using a Binary Decoder and OR Gates – PLD - ROM

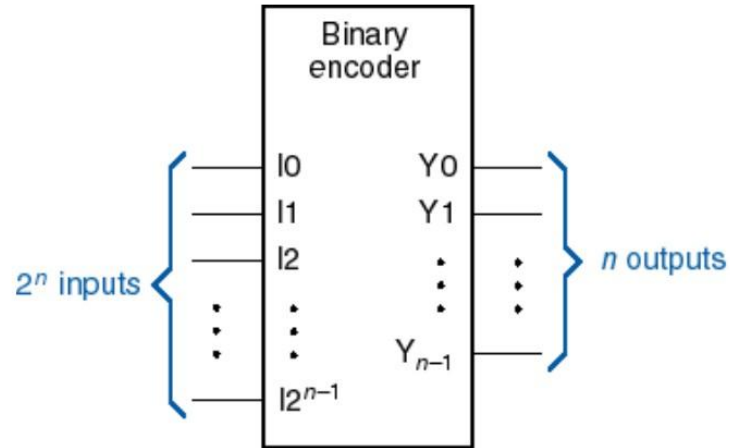
Function

x_2	x_1	x_0	z_2	z_1	z_0
0	0	0	0	1	0
0	0	1	1	0	0
0	1	0	0	0	1
0	1	1	0	1	0
1	0	0	0	0	1
1	0	1	1	0	1
1	1	0	0	1	0
1	1	1	1	0	0



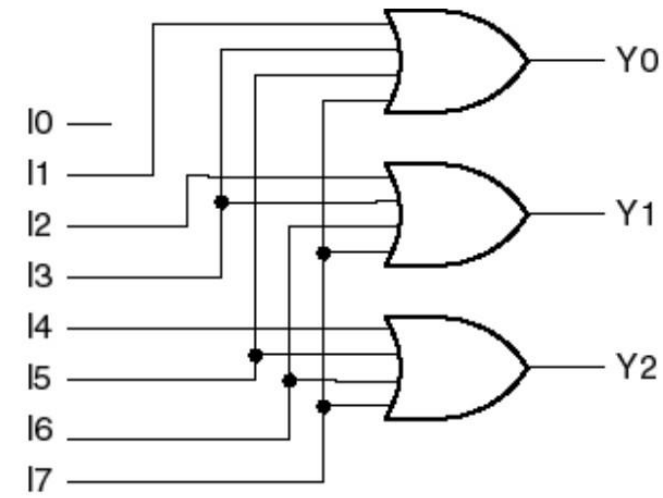
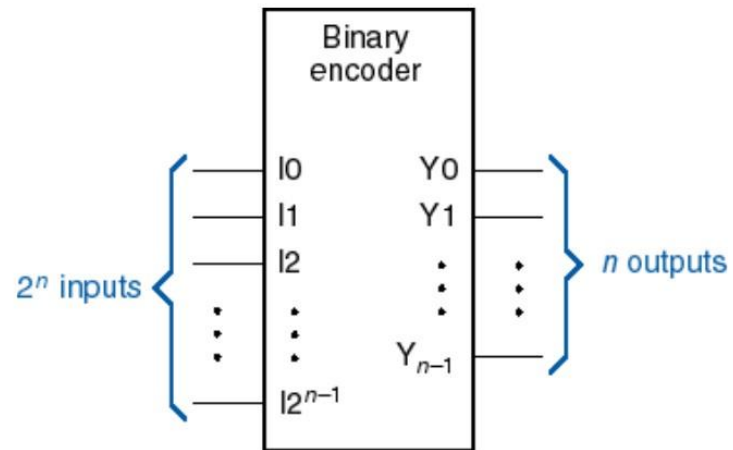
Remember that any function can be represented as a sum of minterms

Binary Encoders



- Only one input I_j has value 1 at any given time
- Output Y corresponds to the binary code of j when $I_j = 1$

8-to-3 Binary Encoders



$$Y_0 = I_1 + I_3 + I_5 + I_7$$

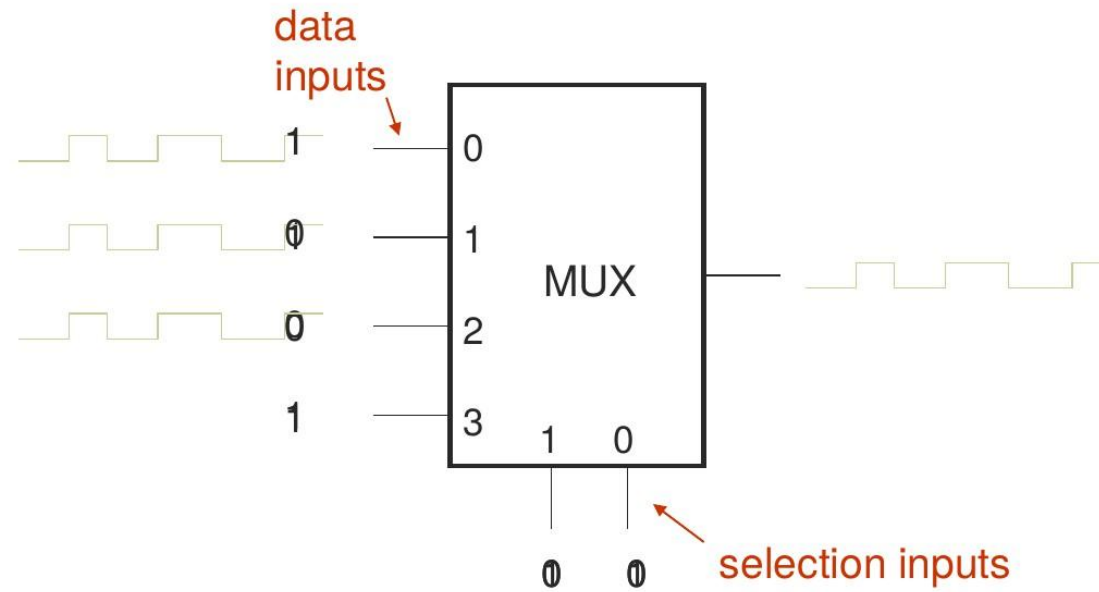
(odd input indices)

$$Y_1 = I_2 + I_3 + I_6 + I_7$$

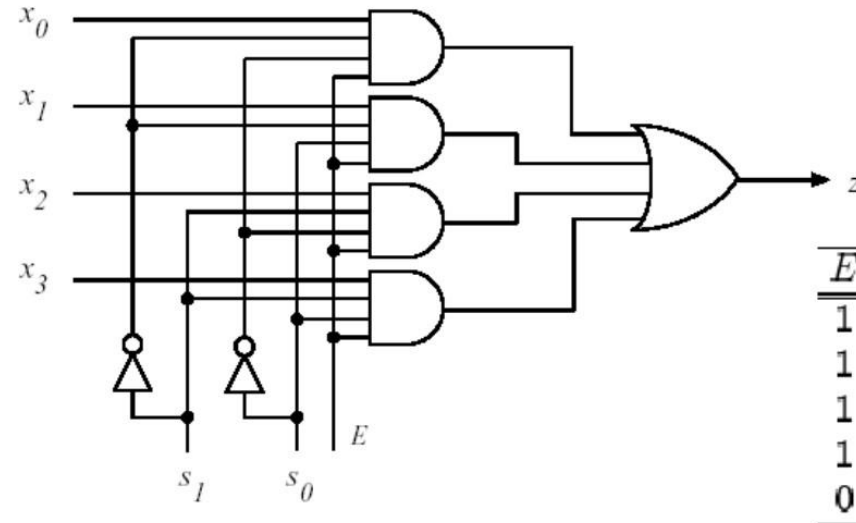
$$Y_2 = I_4 + I_5 + I_6 + I_7$$

(indices > 3)

Multiplexers



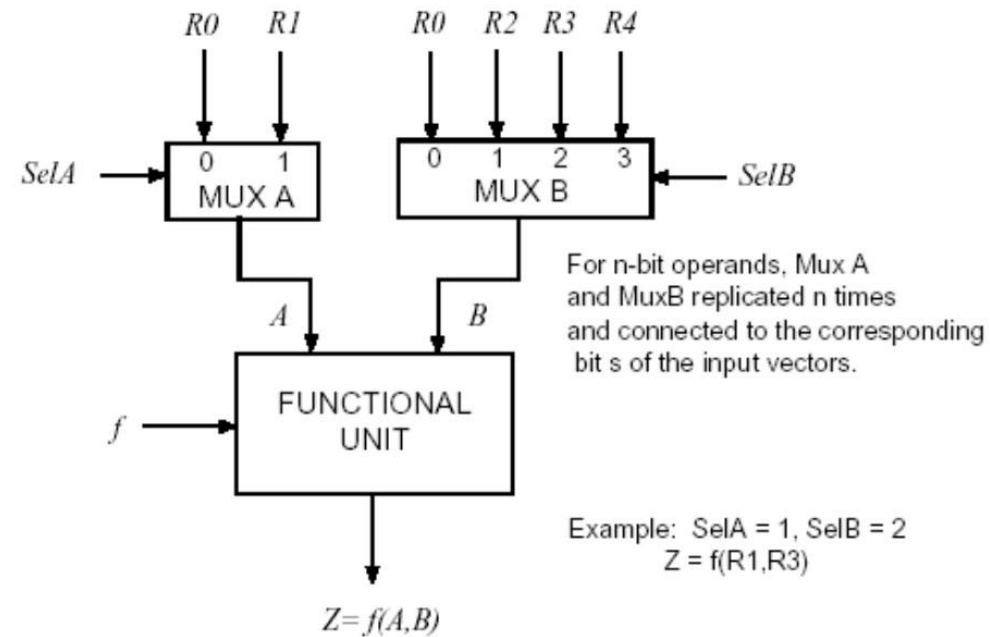
4-input Multiplexer



$$\begin{aligned}
 z &= E \cdot (x_0 m_0(s_1, s_0) + x_1 m_1(s_1, s_0) + x_2 m_2(s_1, s_0) + x_3 m_3(s_1, s_0)) \\
 &= E \cdot (x_0 s_1' s_0' + x_1 s_1' s_0 + x_2 s_1 s_0' + x_3 s_1 s_0)
 \end{aligned}$$

Typical Multiplexer use

selection between multiple paths to a functional unit



Multiplexers as universal modules

- Universal module: using only this module you are able to implement ANY logic function.
- NAND and NOR gates for example are universal gates.
- Question: how do you assign inputs for the multiplexer in order to implement a given function?

Exercise

Implement the following function using:

- a) 8-input multiplexer.
- b) 4-input multiplexer.

$$F = \Sigma_{x,y,z}(1,2,6,7)$$

1-6 Flip-Flops

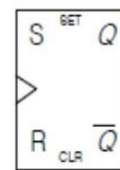
1-24

■ Flip-Flop

- ◆ The *storage elements* employed in clocked *sequential circuit*
- ◆ A binary cell capable of storing one bit of information

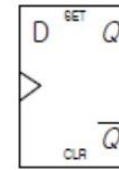
Combinational Circuit = Gate
Sequential Circuit = Gate + F/F

■ SR(Set/Reset) F/F



S	R	Q(t)	Q(t+1)
0	0	Q(t)	no change
0	1	0	clear to 0
1	0	1	set to 1
1	1	?	Indeterminate

■ D(Data) F/F

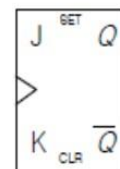


D	Q(t)	Q(t+1)
0	0	clear to 0
1	1	set to 1

- ◆ "no change" condition이 없다 : $Q(t+1)=D$

- 해결 방법 : 1) Disable Clock
2) Feedback output into input p.52

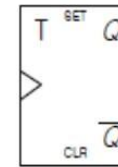
■ JK(Jack/King) F/F



J	K	Q(t)	Q(t+1)
0	0	Q(t)	no change
0	1	0	clear to 0
1	0	1	set to 1
1	1	Q(t)	Complement

- ◆ JK F/F is a refinement of the SR F/F
- ◆ The indeterminate condition of the SR type is defined in complement

■ T(Toggle) F/F



T	Q(t)	Q(t+1)
0	Q(t)	no change
1	Q(t)	Q'(t) Complement

- ◆ $T=1(J=K=1)$, $T=0(J=K=0)$ 이면 JK F/F
- ◆ 수식 표현 : $Q(t+1) = Q(t) \oplus T$ xor

Excitation Table

- ◆ Required input combinations for a given change of state
- ◆ Present State 와 Next State로 표현

SR F/F			
Q(t)	Q(t+1)	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

JK F/F			
Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

D F/F		
Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

T F/F		
Q(t)	Q(t+1)	T
0	0	0
0	1	1
1	0	1
1	1	0

Don't Care

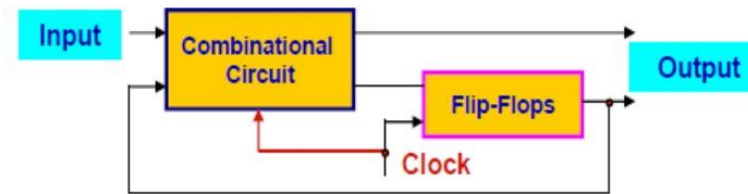
0 : Set to 1
1 : Complement

1 : Clear to 0
0 : No change

1-7 Sequential Circuits

1-27

- A sequential circuit is an interconnection of F/F and Gate
- Clocked synchronous sequential circuit



Combinational Circuit = Gate
Sequential Circuit = Gate + F/F

- Flip-Flop Input Equation

◆ Boolean expression for F/F input

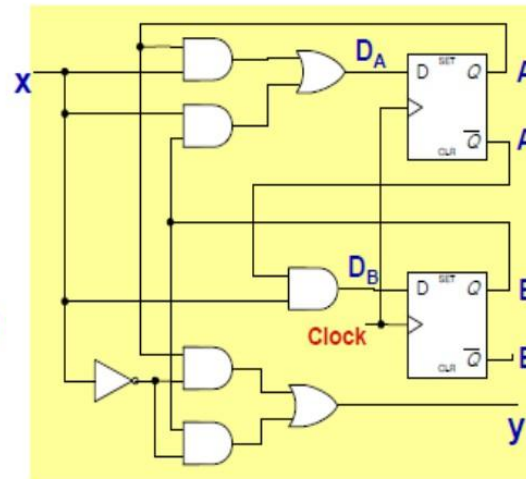
◆ Input Equation 예제

• $D_A = Ax + Bx$, $D_B = A'x$

◆ Output Equation

• $y = Ax' + Bx'$

◆ Fig. 1-25 Example of a sequential circuit



State Table

- Present state, input, next state, output 표현

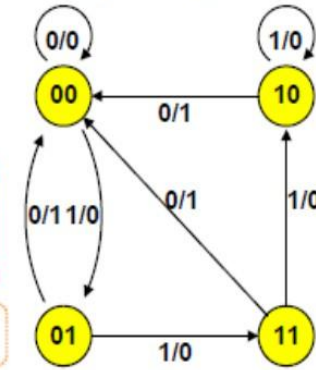
Present State		Input	Input Equ.		Next State		Output
A	B	x	Ax	Bx	A	B	y
0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0
0	1	0	0	0	0	0	1
0	1	1	0	1	1	1	0
1	0	0	0	0	0	0	1
1	0	1	1	0	1	0	0
1	1	0	0	0	0	0	1
1	1	1	1	1	1	0	0

Input Equ. = Next State

State Diagram

- Graphical representation of state table

- Circle(state), Line(transition), I/O(input/output)



JK F/F			
Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Next State = Output

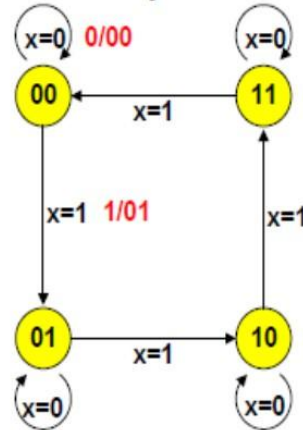
- Excitation Table(2 bit counter = 2 F/F)

Present State		Input	Next State		F/F Input			
A	B	x	A	B	J _A	K _A	J _B	K _B
0	0	0	0	0	0	x	0	x
0	0	1	0	1	0	x	1	x
0	1	0	0	1	0	x	x	0
0	1	1	1	0	1	x	x	1
1	0	0	1	0	x	0	0	x
1	0	1	1	1	x	0	1	x
1	1	0	1	1	x	0	x	0
1	1	1	0	0	x	1	x	1

Design Example: Binary Counter

- x=1: 00, 01, 10, 11, 00, 01,
- x=0: no change

- State Diagram: 4 state(00, 01, 10, 11)



◆ Map for simplification

● Input variable: A, B, x

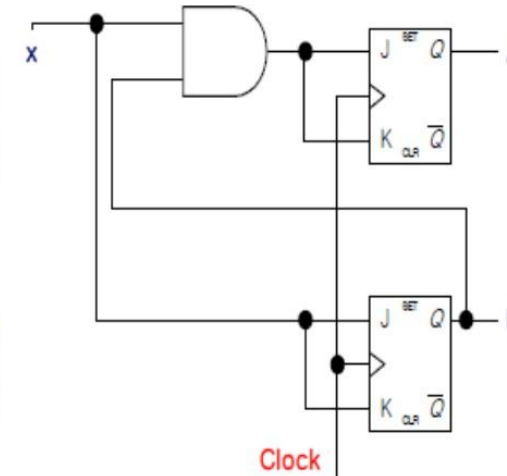
		B			
		0	1	3	2
J _A	A	X	X	1	X
		4	5	7	6
		J _A =Bx			

		B			
		0	1	3	2
K _A	A	X	X	X	X
		4	5	7	6
		K _A =Bx			

		B			
		0	1	3	2
J _B	A	1	X	X	X
		4	1	X	X
		J _B =x			

		B			
		0	1	3	2
K _B	A	X	X	1	X
		4	X	1	X
		K _B =x			

◆ Logic Diagram

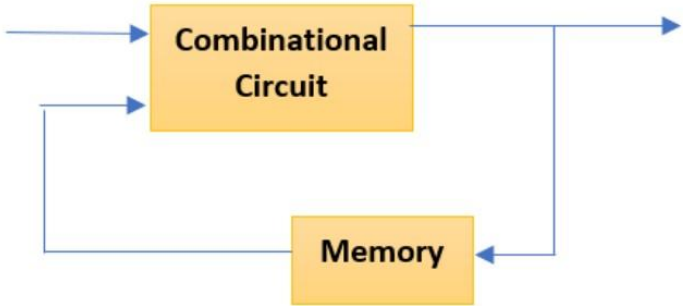


■ Sequential Circuit Design Procedure

- ◆ 1-5 절 참고(Combinational Circuit Design)
- ◆ Sequential Circuit은 절차 3에서 State diagram 및 State table 이용
- ◆ # of rows : 2^{m+n} (m - State 수, n - Input 수)

1. The Problem is stated
2. I/O variables are assigned
3. Truth table(I/O relation)
4. Simplified Boolean Function
5. Logic circuit diagram

Combinational circuit vs sequential circuits

Combinational Circuit	Sequential Circuit
Output only depends on the present input	Output depends on present input and past output
Memory element is absent	Memory element is present
No clock signal is applied	Clock signal is required
	
Example - Half Adder, Full Adder, Multiplexer	Examples - Flipflop, Counters, Registers

Types of sequential circuits

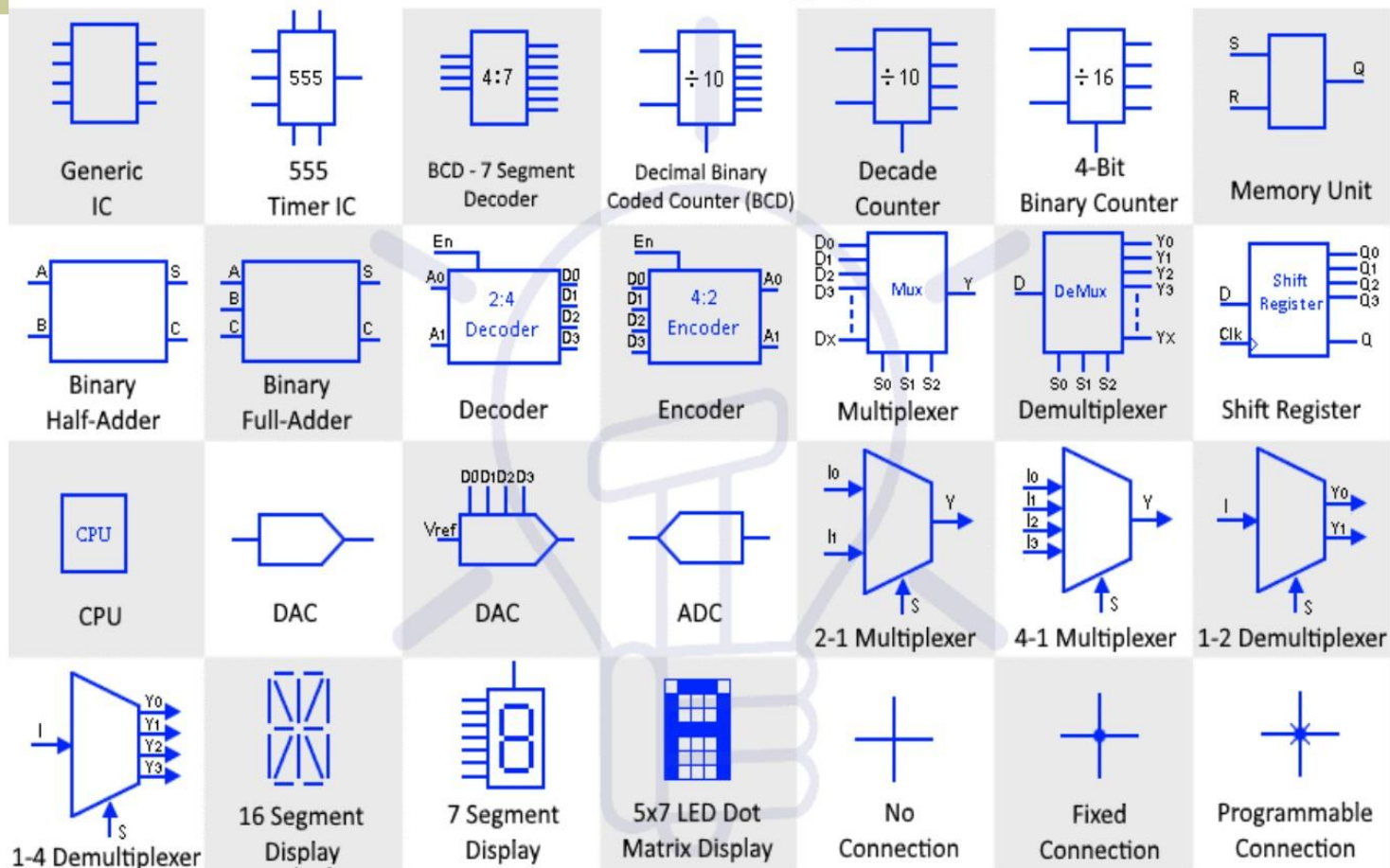
There are two types of sequential circuits: Asynchronous and synchronous sequential circuits.

Asynchronous sequential circuit

- Asynchronous sequential circuits are driven by the pulses of the inputs instead of using the clock signals, that is the state of circuit changes when the inputs change. The change in internal state occurs when there is change in the input variable. Un-clocked flip flops or time delay elements are used as memory elements.
- **Synchronous sequential circuit**
- In synchronous sequential circuits, the state of memory changes with the clock signal connected. That is, all the outputs are affected by the clock signal at the same time. Here, flip flops and latches are used as memory elements.

Logic Circuits & Digital Programming Symbols

www.electricaltechnology.org



CAS202 Computer Organization (1)

10/3/2026

41

Digital logic design

Digital logic design relies on fundamental electronic switches, basic logic gates, combinational building blocks, and sequential memory devices.

Transistors and Basic Logic Gates

- The core hardware foundation starts with transistors acting as binary switches (0 and 1), which combine to form the foundational logic gates:
- **NOT Gate (Inverter):** Reverses the input binary state (0 becomes 1, and 1 becomes 0).
- **AND Gate:** Outputs 1 only if all inputs are 1.
- **OR Gate:** Outputs 1 if at least one input is 1.
- **NAND Gate:** Inverted AND gate; outputs 0 only if all inputs are 1.
- **NOR Gate:** Inverted OR gate; outputs 1 only if all inputs are 0.
- **XOR Gate (Exclusive-OR):** Outputs 1 if inputs are different.
- **XNOR Gate (Exclusive-NOR):** Outputs 1 if inputs are the same.

Digital logic design

Combinational Logic Components

- Circuits whose outputs depend strictly on the current inputs without any memory or past states:
- **Adders and Subtractors:** Arithmetic blocks like half-adders and full-adders that compute binary math.
- **Multiplexers (MUX):** Selects one of several input data lines and forwards it to a single output line.
- **Demultiplexers (DEMUX):** Takes a single input signal and routes it to one of multiple outputs.
- **Encoders and Decoders:** Converts data formats, such as binary to decimal or BCD to a seven-segment display.
- **Comparators:** Compares the magnitude of two binary numbers.
- **Arithmetic Logic Unit (ALU):** Executes basic arithmetic and bitwise logical operations inside processors.

Digital logic design

Sequential Logic Devices

- Circuits that include memory elements where the output depends on both current inputs and previous states: [1]
- **Latches and Flip-Flops:** Basic 1-bit memory elements (SR, D, JK, and T flip-flops).
- **Registers:** Groups of flip-flops used to store multiple bits of data.
- **Counters:** Sequential circuits that count clock pulses (up, down, or ripple counters).
- **Memory Units:** RAM (Random Access Memory) and ROM (Read-Only Memory).
- **Finite State Machines (FSM):** Complex controllers that transition between predefined states based on inputs.

Digital logic design

Programmable and Integrated Devices

- **Integrated Circuits (ICs):** Silicon chips combining multiple logic gates or systems into a single microelectronic package.
- **PLDs, PALs, and PLAs:** Programmable logic devices configured by users to implement custom logic equations.
- **FPGAs (Field-Programmable Gate Arrays):** High-density semiconductor devices containing programmable logic blocks for complex digital systems.
- **HOW TO DESIGN** a specific component, such as a **finite state machine** or an **arithmetic logic unit (ALU)**, using **Boolean algebra** or a hardware description language like **Verilog & VHDL?**



Lab Tutorial 1

Chapter (1)



Logic Gates

Logic Gates

The Inverter

The AND Gate

The OR Gate

The NAND Gate

The NOR Gate

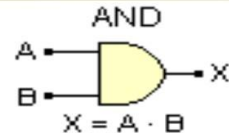
The XOR Gate

The XNOR Gate

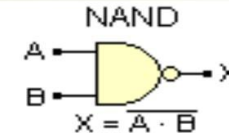
The Buffer Gate

Drawing Logic Circuit

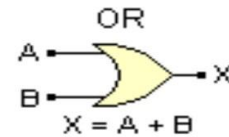
Analysing Logic Circuit



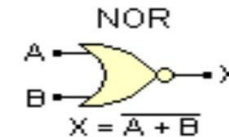
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1



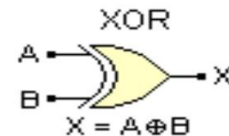
A	B	X
0	0	1
0	1	1
1	0	1
1	1	0



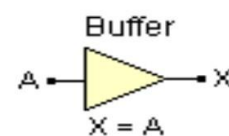
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1



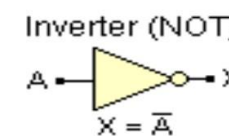
A	B	X
0	0	1
0	1	0
1	0	0
1	1	0



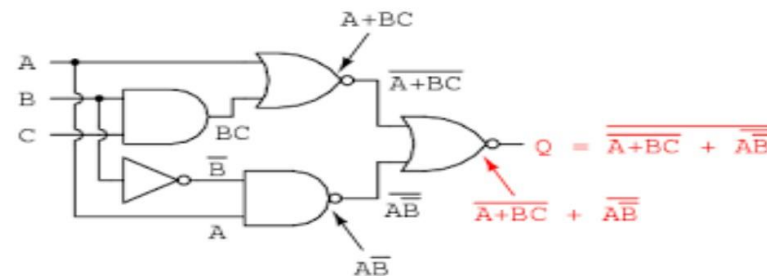
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0



A	X
0	0
1	1



A	X
0	1
1	0



Boolean Algebra

- Boolean Function: variable + operation

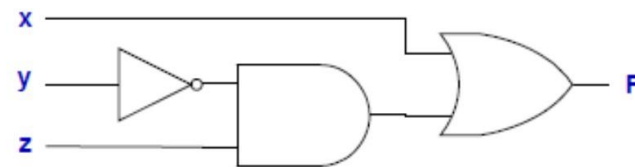
◆ $F(x, y, z) = x + y'z$

- Truth Table: *Fig. 1-3(a)*
Relationship between a function and variable

2ⁿ Combination
Variable n = 3

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

- Logic Diagram: *Fig. 1-3(b)*
Algebraic Expression $\rightarrow$
Logic Diagram(gates로 표현)



Boolean Algebra Rule

■ Boolean Algebra Rule :

- **Operation with 0 and 1:** $x + 0 = x$, $x + 1 = 1$, $x \cdot 1 = x$, $x \cdot 0 = 0$

- **Idempotent Law:** $x + x = x$, $x \cdot x = x$

- **Complementary Law:** $x + x' = 1$, $x \cdot x' = 0$

- **Commutative Law:** $x + y = y + x$, $x \cdot y = y \cdot x$

- **Associative Law:** $x + (y + z) = (x + y) + z$, $x \cdot (y \cdot z) = (x \cdot y) \cdot z$

- **Distributive Law:** $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$, $x + (y \cdot z) = (x + y) \cdot (x + z)$

- **DeMorgan's Law:** $(x + y)' = x' \cdot y'$, $(x \cdot y)' = x' + y'$

General Form: $(x_1 + x_2 + x_3 + \dots x_n)' = x_1' \cdot x_2' \cdot x_3' \cdot \dots x_n'$

$(x_1 \cdot x_2 \cdot x_3 \cdot \dots x_n)' = x_1' + x_2' + x_3' + \dots x_n'$

$$(A \cup B)^c = A^c \cap B^c$$

Map Simplification

Map

◆ 2 variables

B

0	1
2	3

A

◆ 3 variables

B

0	1	3	2
4	5	7	6

C

◆ 4 variables

C

0	1	3	2
4	5	7	6
12	13	15	14
8	9	11	10

A

D

◆ 5 variables

C

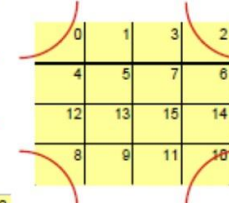
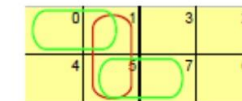
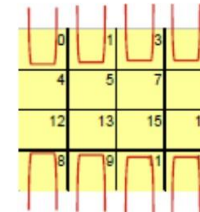
0	1	3	2	6	7	5	4
8	9	11	10	14	15	13	12
24	25	27	26	30	31	29	28
16	17	19	18	22	23	21	20

E *D* *F*

Map Simplification

■ Adjacent Square

- ◆ Number of square = 2^n (2, 4, 8,)
- ◆ The squares at the extreme ends of the same horizontal row are to be considered adjacent
- ◆ The same applies to the top and bottom squares of a column
- ◆ The four corner squares of a map must be considered to be adjacent
- ◆ Groups of combined adjacent squares may share one or more squares with one or more group



Map Simplification

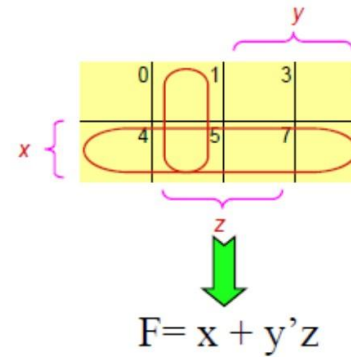
■ $F = x + y'z$

(1) Truth Table

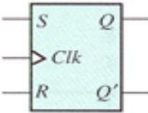
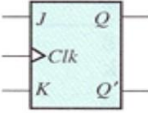
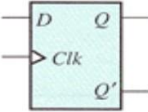
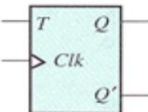
x	y	z	F	Minterm
0	0	0	0	m_0
0	0	1	1	m_1
0	1	0	0	m_2
0	1	1	0	m_3
1	0	0	1	m_4
1	0	1	1	m_5
1	1	0	1	m_6
1	1	1	1	m_7

(2) $F(x, y, z) = \Sigma(1, 4, 5, 6, 7)$

(3)



Flip-Flops

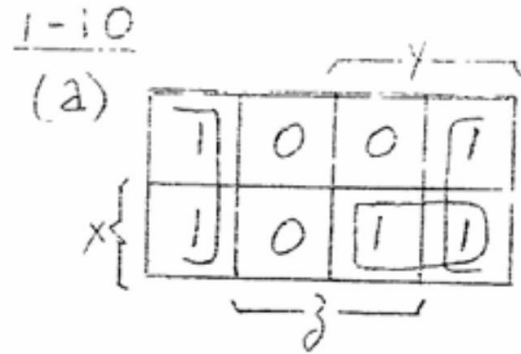
FLIP-FLOP NAME	FLIP-FLOP SYMBOL	CHARACTERISTIC TABLE	CHARACTERISTIC EQUATION	EXCITATION TABLE																																			
SR		<table> <tr> <th>S</th> <th>R</th> <th>Q(next)</th> </tr> <tr> <td>0</td> <td>0</td> <td>Q</td> </tr> <tr> <td>0</td> <td>1</td> <td>0</td> </tr> <tr> <td>1</td> <td>0</td> <td>1</td> </tr> <tr> <td>1</td> <td>1</td> <td>NA</td> </tr> </table>	S	R	Q(next)	0	0	Q	0	1	0	1	0	1	1	1	NA	$Q(next) = S + R'Q$ $SR = 0$	<table> <tr> <th>Q</th> <th>Q(next)</th> <th>S</th> <th>R</th> </tr> <tr> <td>0</td> <td>0</td> <td>0</td> <td>X</td> </tr> <tr> <td>0</td> <td>1</td> <td>1</td> <td>0</td> </tr> <tr> <td>1</td> <td>0</td> <td>0</td> <td>1</td> </tr> <tr> <td>1</td> <td>1</td> <td>X</td> <td>0</td> </tr> </table>	Q	Q(next)	S	R	0	0	0	X	0	1	1	0	1	0	0	1	1	1	X	0
S	R	Q(next)																																					
0	0	Q																																					
0	1	0																																					
1	0	1																																					
1	1	NA																																					
Q	Q(next)	S	R																																				
0	0	0	X																																				
0	1	1	0																																				
1	0	0	1																																				
1	1	X	0																																				
JK		<table> <tr> <th>J</th> <th>K</th> <th>Q(next)</th> </tr> <tr> <td>0</td> <td>0</td> <td>Q</td> </tr> <tr> <td>0</td> <td>1</td> <td>0</td> </tr> <tr> <td>1</td> <td>0</td> <td>1</td> </tr> <tr> <td>1</td> <td>1</td> <td>Q'</td> </tr> </table>	J	K	Q(next)	0	0	Q	0	1	0	1	0	1	1	1	Q'	$Q(next) = JQ' + K'Q$	<table> <tr> <th>Q</th> <th>Q(next)</th> <th>J</th> <th>K</th> </tr> <tr> <td>0</td> <td>0</td> <td>0</td> <td>X</td> </tr> <tr> <td>0</td> <td>1</td> <td>1</td> <td>X</td> </tr> <tr> <td>1</td> <td>0</td> <td>X</td> <td>1</td> </tr> <tr> <td>1</td> <td>1</td> <td>X</td> <td>0</td> </tr> </table>	Q	Q(next)	J	K	0	0	0	X	0	1	1	X	1	0	X	1	1	1	X	0
J	K	Q(next)																																					
0	0	Q																																					
0	1	0																																					
1	0	1																																					
1	1	Q'																																					
Q	Q(next)	J	K																																				
0	0	0	X																																				
0	1	1	X																																				
1	0	X	1																																				
1	1	X	0																																				
D		<table> <tr> <th>D</th> <th>Q(next)</th> </tr> <tr> <td>0</td> <td>0</td> </tr> <tr> <td>1</td> <td>1</td> </tr> </table>	D	Q(next)	0	0	1	1	$Q(next) = D$	<table> <tr> <th>Q</th> <th>Q(next)</th> <th>D</th> </tr> <tr> <td>0</td> <td>0</td> <td>0</td> </tr> <tr> <td>0</td> <td>1</td> <td>1</td> </tr> <tr> <td>1</td> <td>0</td> <td>0</td> </tr> <tr> <td>1</td> <td>1</td> <td>1</td> </tr> </table>	Q	Q(next)	D	0	0	0	0	1	1	1	0	0	1	1	1														
D	Q(next)																																						
0	0																																						
1	1																																						
Q	Q(next)	D																																					
0	0	0																																					
0	1	1																																					
1	0	0																																					
1	1	1																																					
T		<table> <tr> <th>T</th> <th>Q(next)</th> </tr> <tr> <td>0</td> <td>Q</td> </tr> <tr> <td>1</td> <td>Q'</td> </tr> </table>	T	Q(next)	0	Q	1	Q'	$Q(next) = TQ' + T'Q$	<table> <tr> <th>Q</th> <th>Q(next)</th> <th>T</th> </tr> <tr> <td>0</td> <td>0</td> <td>0</td> </tr> <tr> <td>0</td> <td>1</td> <td>1</td> </tr> <tr> <td>1</td> <td>0</td> <td>1</td> </tr> <tr> <td>1</td> <td>1</td> <td>0</td> </tr> </table>	Q	Q(next)	T	0	0	0	0	1	1	1	0	1	1	1	0														
T	Q(next)																																						
0	Q																																						
1	Q'																																						
Q	Q(next)	T																																					
0	0	0																																					
0	1	1																																					
1	0	1																																					
1	1	0																																					

Problems



- 1-10. Simplify the following expressions in (1) sum-of-products form and (2) product-of-sums form.
- a. $x'z' + y'z' + yz' + xy$
 - b. $AC' + B'D + A'CD + ABCD$

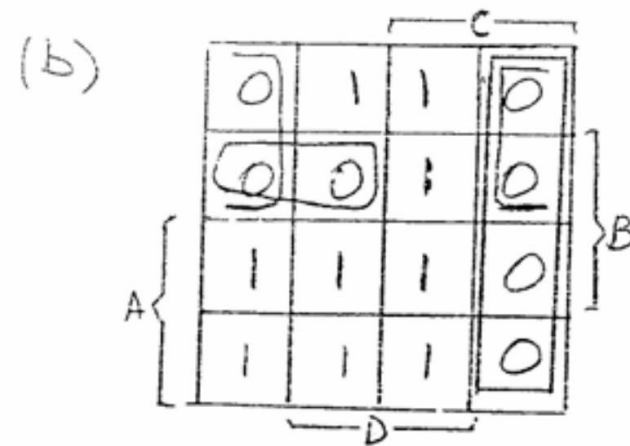
- 1-10. Simplify the following expressions in (1) sum-of-products form and (2) product-of-sums form.
- $x'z' + y'z' + yz' + xy$
 - $AC' + B'D + A'CD + ABCD$



$$(1) F = xy + z'$$

$$F' = x'z + y'z$$

$$(2) F = (x + z')(y + z')$$



$$(1) F = AC' + CD + B'D$$

$$(2) F = (A + D)(C' + D)(A + B' + C)$$

- 1-13. Simplify the Boolean function F together with the don't-care conditions d in (1) sum-of-products form and (2) product-of-sums form.

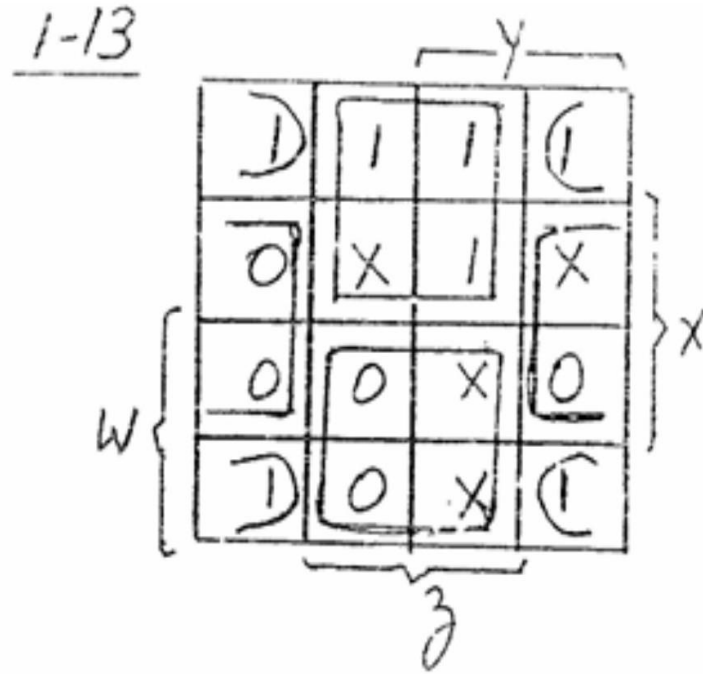
$$F(w, x, y, z) = \sum (0, 1, 2, 3, 7, 8, 10)$$

$$d(w, x, y, z) = \sum (5, 6, 11, 15)$$

- 1-13. Simplify the Boolean function F together with the don't-care conditions d in (1) sum-of-products form and (2) product-of-sums form.

$$F(w, x, y, z) = \sum (0, 1, 2, 3, 7, 8, 10)$$

$$d(w, x, y, z) = \sum (5, 6, 11, 15)$$



$$(a) F = x'z' + w'z$$

$$(b) = (x' + z)(w' + z')$$

- 1-14. Using Table 1-2, derive the Boolean expression for the S (sum) output of the full-adder in sum-of-products form. Then by algebraic manipulation show that S can be expressed as the exclusive-OR of the three input variables.

$$S = x \oplus y \oplus z$$

TABLE 1-2 Truth Table for Full-Adder

Inputs			Outputs	
x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

- 1-14. Using Table 1-2, derive the Boolean expression for the S (sum) output of the full-adder in sum-of-products form. Then by algebraic manipulation show that S can be expressed as the exclusive-OR of the three input variables.

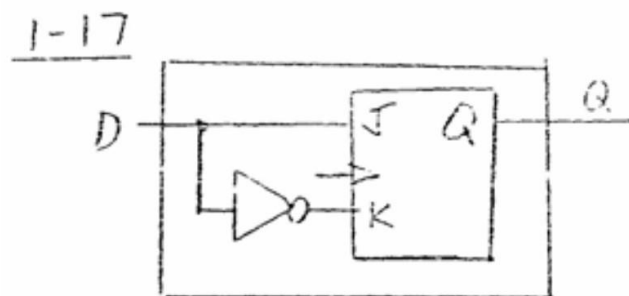
$$S = x \oplus y \oplus z$$

1-14

$$\begin{aligned}
 S &= x'y'z + x'yz' + xy'z' + xyz \\
 &= x'(y'z + yz') + x(y'z' + yz) \quad \leftarrow \text{See Fig 1-2} \\
 &= x'(y \oplus z) + x(y \oplus z)' \quad \leftarrow \text{(Exclusive-NOR)} \\
 &= x \oplus y \oplus z
 \end{aligned}$$

- 1-17. Show that a JK flip-flop can be converted to a D flip-flop with an inverter between the J and K inputs.

- 1-17. Show that a JK flip-flop can be converted to a D flip-flop with an inverter between the J and K inputs.



When $D=0$; $J=0, K=1, Q \rightarrow 0$

When $D=1$; $J=1, K=0, Q \rightarrow 1$

1-19. A sequential circuit has two D flip-flops A and B , two inputs x and y , and one output z . The flip-flop input equations and the circuit output are as follows:

$$D_A = x'y + xA$$

$$D_B = x'B + xA$$

$$z = B$$

- Draw the logic diagram of the circuit.
- Tabulate the state table.

1-19. A sequential circuit has two D flip-flops A and B, two inputs x and y, and one output z. The flip-flop input equations and the circuit output are as follows:

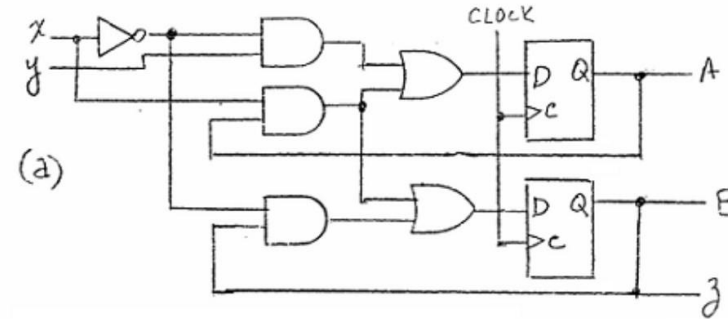
$$D_A = x'y + xA$$

$$D_B = x'B + xA$$

$$z = B$$

- Draw the logic diagram of the circuit.
- Tabulate the state table.

1-19 $D_A = x'y + xA$; $D_B = x'B + xA$; $z = B$



(b)

Present state		Inputs		Next state		Output
A	B	x	y	A	B	z
0	0	0	0	0	0	0
0	0	0	1	1	0	0
0	0	1	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	1	1
0	1	0	1	1	1	1
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	0	0	0
1	0	0	1	1	0	0
1	0	1	0	1	1	0
1	0	1	1	1	1	0
1	1	0	0	0	1	1
1	1	0	1	1	1	1
1	1	1	0	1	1	1
1	1	1	1	1	1	1



Lab Tutorial 2

Chapter (2)



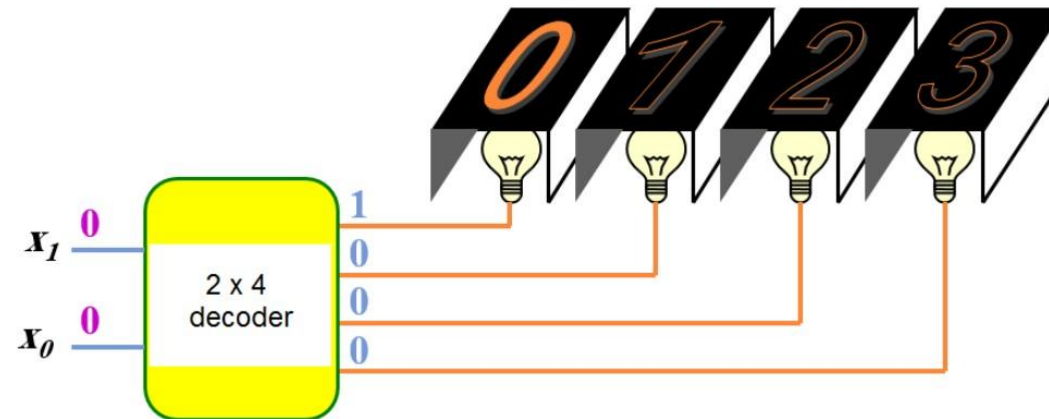
Agenda

- Decoder
- Encoder
- Multiplexer
- Register
- Shift Register
- Memory Unit

Decoder

■ Decoder

- ◆ A combinational circuit that converts binary information from the n coded inputs to a maximum of 2^n unique outputs
- ◆ n -to- m line decoder = $n \times m$ decoder
 - n inputs, m outputs
- ◆ If the n -bit coded information has unused bit combinations, the decoder may have less than 2^n outputs
 - $m \leq 2^n$

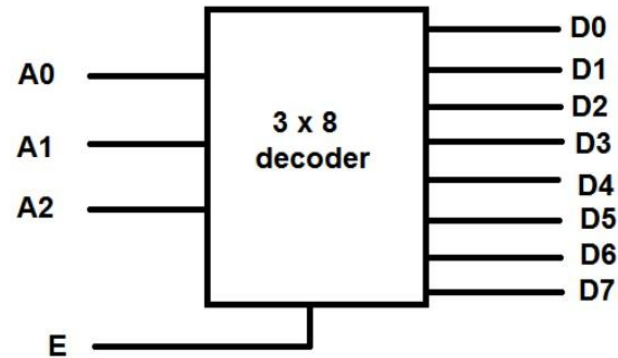


3-to-8 Decoder

- ◆ Logic Diagram : *Fig. 2-1*
- ◆ Truth Table : *Tab. 2-1*
- ◆ Commercial decoders include one or more Enable Input(E)

Enable	Inputs			Outputs							
E	A2	A1	A0	D7	D6	D5	D4	D3	D2	D1	D0
0	x	x	x	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

Tab. 2-1 Truth table for 3-to-8 Decoder



3-to-8 Decoder

Decoder Expansion

- Decoder Expansion
 - ◆ 3 X 8 Decoder constructed with two 2 X 4 Decoder
 - ◆ Constructed decoder : *Fig. 2-3*

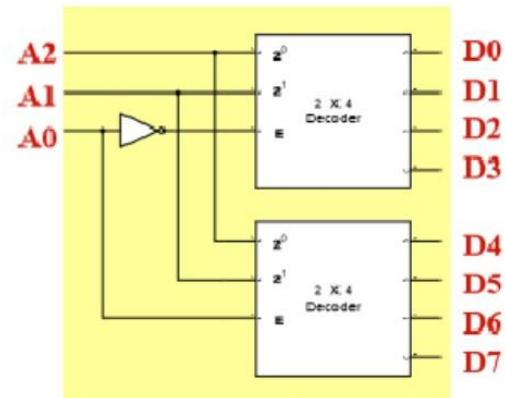


Fig. 2-3 A 3-to-8 Decoder constructed with two with 2-to-4 Decoder

Encoder

Encoder

- ◆ Inverse Operation of a decoder
- ◆ 2^n input, n output
- ◆ Truth Table : *Tab. 2-2*
 - 3 OR Gates Implementation
 - » $A0 = D1 + D3 + D5 + D7$
 - » $A1 = D2 + D3 + D6 + D7$
 - » $A2 = D4 + D5 + D6 + D7$

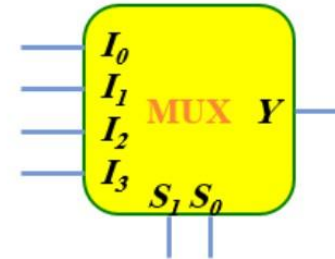
Tab. 2-2 Truth Table for Encoder

Inputs								Outputs		
D7	D6	D5	D4	D3	D2	D1	D0	A2	A1	A0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

Multiplexer (MUX)

■ Multiplexer(Mux)

- ◆ A combinational circuit that receives binary information from one of 2^n input data lines and directs it to a single output line
- ◆ A 2^n -to 1 multiplexer has 2^n *input data lines* and n *input selection lines*(Data Selector)
- ◆ 4-to-1 multiplexer Diagram : *Fig. 2-4*



- ◆ 4-to-1 multiplexer Function Table : *Tab. 2-3*

*Tab. 2-3 Function Table for
4-to-1 line Multiplexer*

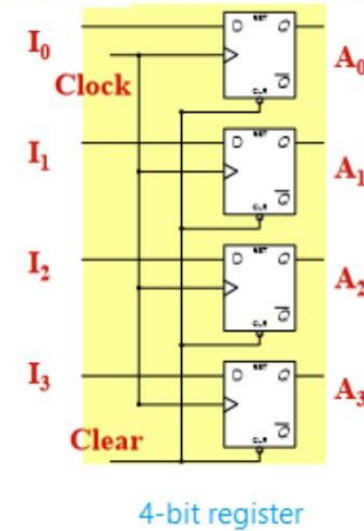
Select		Output
S1	S0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Register

■ Register

- ◆ A group of flip-flops with each flip-flop capable of storing one bit of information
- ◆ An n-bit register has a group of n flip-flops and is capable of storing any binary information of n bits
- ◆ The simplest register consists only of flip-flops, with no external gate :

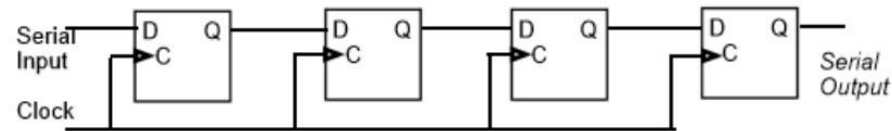
Fig. 2-6



Shift Register

■ Shift Register

- ◆ A register capable of shifting its binary information in one or both directions
- ◆ The logical configuration of a shift register consists of a chain of flip-flops in cascade
- ◆ The simplest possible shift register uses only flip-flops : *Fig. 2-8*
- ◆ The **serial input** determines what goes into the leftmost position during the shift
- ◆ The **serial output** is taken from the output of the rightmost flip-flop



4-bit shift register

Shift Register

- Bidirectional Shift Register with Parallel Load
 - ◆ A register capable of shifting in *one direction only* is called a ***unidirectional shift register***
 - ◆ A register that can shift in *both directions* is called a ***bidirectional shift register***

Memory Unit

■ Memory Unit

- ◆ A collection of storage cells together with associated circuits needed to transfer information in and out of storage
- ◆ The memory stores binary information in groups of bits called **words**
- ◆ Word
 - A group of binary information that is processed in one simultaneous operation
- ◆ Byte
 - A group of eight bits (nibble : four bits)
- ◆ The number of address line = k
 - Address(Identification number) : 0, 1, 2, 3, ... up to $2^k - 1$
 - The selection of specific word inside memory : k bit binary address
 - 1 Kilo= 2^{10} , 1 Mega= 2^{20} , 1 Giga= 2^{30}

Content of a memory

- Each **word in memory** is assigned an **identification number**, called an **address**, starting **from 0 up to 2^k-1** , where **k** is the **number of address lines**.

- The number of words in a memory with one of the letters $K=2^{10}$, $M=2^{20}$, or $G=2^{30}$.

$$64K = 2^{16} \quad 2M = 2^{21}$$

$$4G = 2^{32}$$

Memory address		Memory content
Binary	decimal	
000000000	0	1011010101011101
000000001	1	1010101110001001
000000010	2	0000110101000110
	⋮	⋮
111111101	1021	1001110100010100
111111110	1022	0000110100011110
111111111	1023	1101111000100101

Content of a 1024×16 Memory

Random Access Memory

■ Random Access Memory

- ◆ The memory cells can be accessed for information transfer from any desired random location
- ◆ Communication between a memory and its environment is achieved through **data input and output lines**, **address selection lines**, and **control lines** : *Fig. 2-12*
- ◆ The two operations that a random-access memory can perform are the **write** and **read** operations

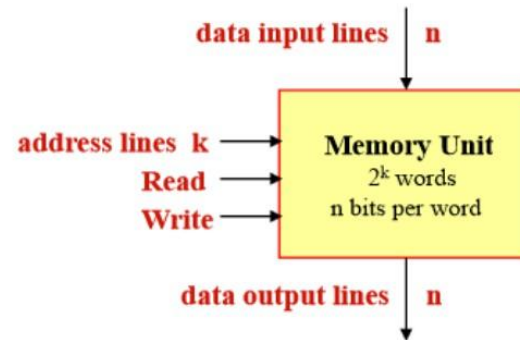
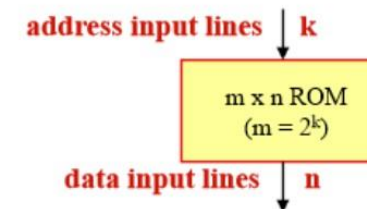


Fig. 2-12 Block diagram of RAM

Read Only Memory

■ Read-Only Memory

- ◆ A memory unit that performs the read operation **only**; it does not have a write capability
- ◆ $m \times n$ ROM :
 - k address input lines to select one of $2^k = m$ words of memory, and n output lines (n bits word)



- ◆ ROM is classified as a **combinational circuit**, because the outputs are a function of only the present inputs (address lines)

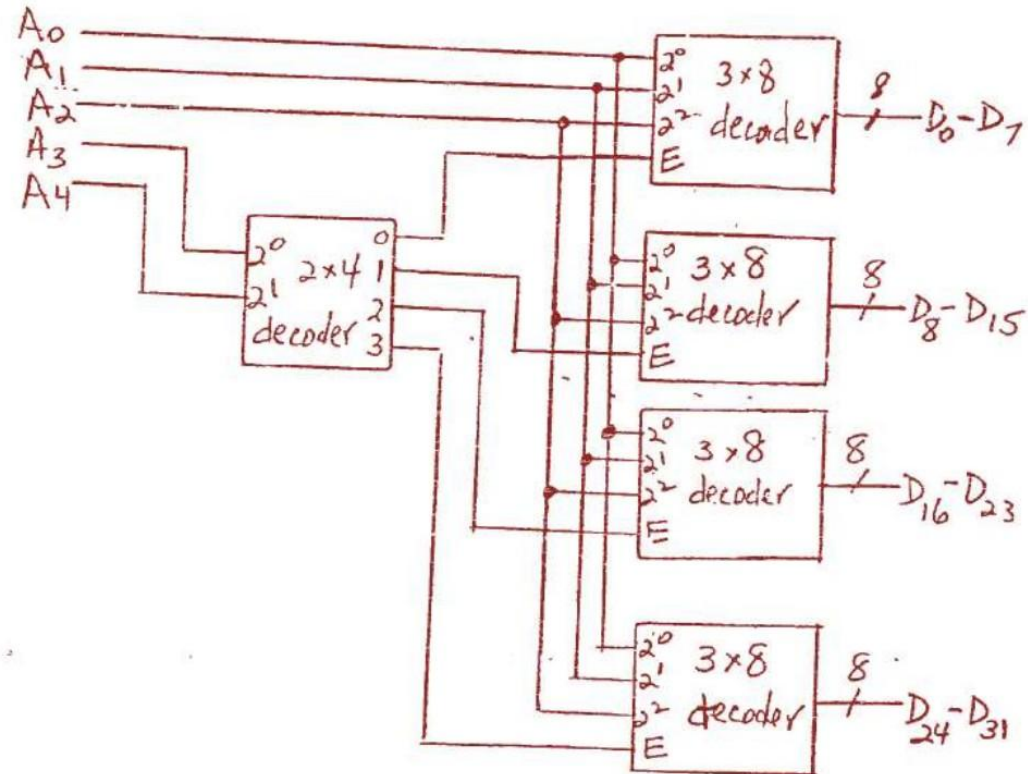
Problems



- 2-3. Construct a 5-to-32-line decoder with four 3-to-8-line decoders with enable and one 2-to-4-line decoder. Use block diagrams similar to Fig. 2-3.

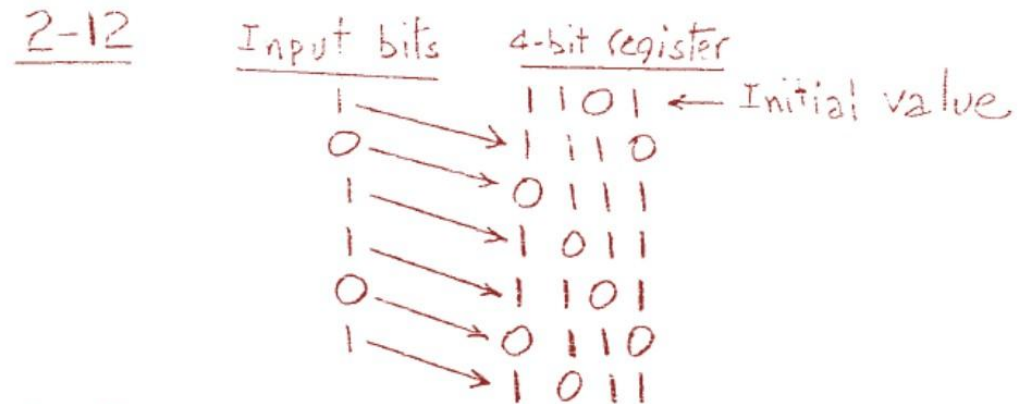
- 2-3. Construct a 5-to-32-line decoder with four 3-to-8-line decoders with enable and one 2-to-4-line decoder. Use block diagrams similar to Fig. 2-3.

2-3



2-12. The content of a 4-bit register is initially 1101. The register is shifted six times to the right with the serial input being 101101. What is the content of the register after each shift?

- 2-12. The content of a 4-bit register is initially 1101. The register is shifted six times to the right with the serial input being 101101. What is the content of the register after each shift?



2-13. What is the difference between serial and parallel transfer? Using a shift register with parallel load, explain how to convert serial input data to parallel output and parallel input data to serial output.

- 2-13. What is the difference between serial and parallel transfer? Using a shift register with parallel load, explain how to convert serial input data to parallel output and parallel input data to serial output.

2-13

Serial transfer: One bit at a time by shifting.

Parallel transfer: All bits at the same time.

Input serial data by shifting - output data in parallel.

Input data with parallel load - Output data by shifting.

- 2-19. The following memory units are specified by the number of words times the number of bits per word. How many address lines and input–output data lines are needed in each case? (a) $2\text{K} \times 16$; (b) $64\text{K} \times 8$; (c) $16\text{M} \times 32$; (d) $4\text{G} \times 64$.

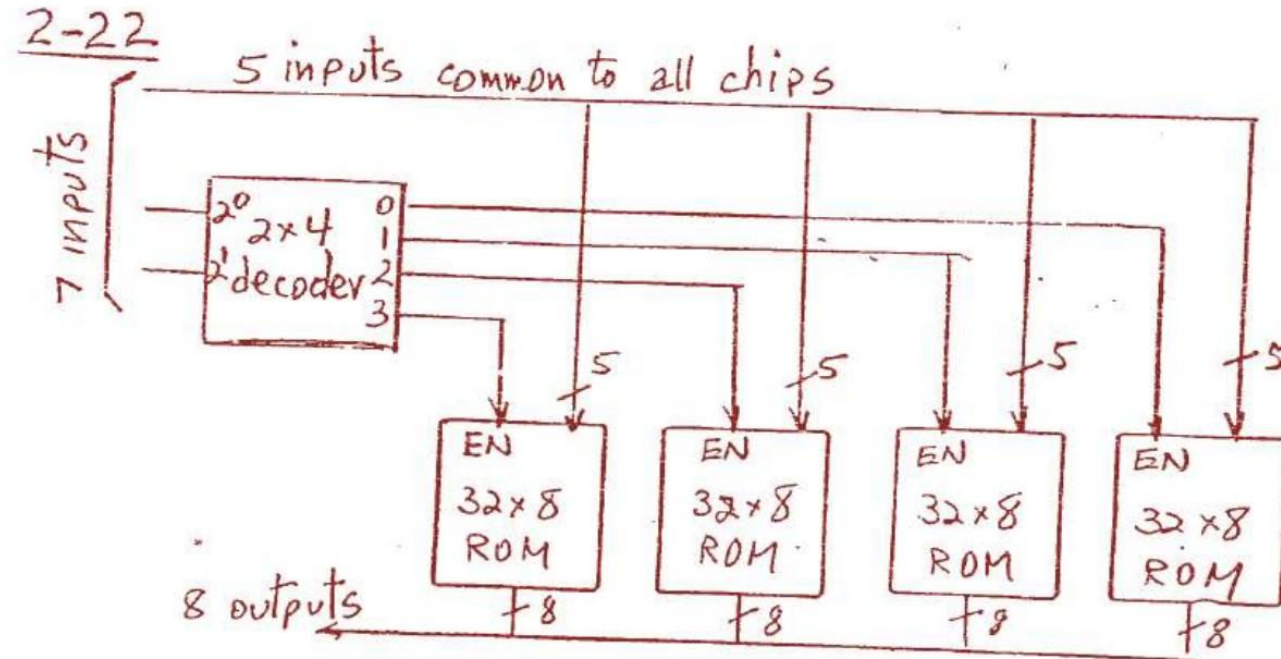
- 2-19. The following memory units are specified by the number of words times the number of bits per word. How many address lines and input-output data lines are needed in each case? (a) $2K \times 16$; (b) $64K \times 8$; (c) $16M \times 32$; (d) $4G \times 64$.

2-19

	Address lines	Data lines
(a) $2K \times 16 = 2^{11} \times 16$	11	16
(b) $64K \times 8 = 2^{16} \times 8$	16	8
(c) $16M \times 32 = 2^{24} \times 32$	24	32
(d) $4G \times 64 = 2^{32} \times 64$	32	64

- 2-22. Given a 32×8 ROM chip with an enable input, show the external connections necessary to construct a 128×8 ROM with four chips and a decoder.

- 2-22. Given a 32×8 ROM chip with an enable input, show the external connections necessary to construct a 128×8 ROM with four chips and a decoder.





Thank You

